UNIVERSIDAD DEL VALLE DE GUATEMALA

Laboratorio 12

Cálculo lambda y programación funcional

Curso	Teoría de la Computación
Entrega	Noviembre de 2025
Lenguaje	JavaScript (Node.js), con funciones lambda
Contenido	Problema 1: respuestas teóricas y demostracion de NOT

Objetivo

Presentar la reducción-beta de la negación lógica y explicar cómo se representan la recursión y los ciclos bajo el paradigma funcional, incluyendo criterios concretos para decidir cuando conviene utilizarlo.

Nota: el repositorio incluye una implementacion ejecutable de los booleanos de Church para relacionar esta explicacion con la demostracion solicitada en video.

PROBLEMA 1 · 25%

Reducción-beta de la operación lógica NOT

En cálculo lambda, un booleano no necesita ser un valor incorporado. Puede representarse como una función que elige entre dos argumentos. Estas son las codificaciones de Church:

```
TRUE  ≡ λx.λy.x
FALSE ≡ λx.λy.y
NOT   ≡ λp.p FALSE TRUE
```

La expresión NOT entrega a la proposición primero FALSE y despues TRUE. Si la proposición es TRUE, escoge el primer argumento (FALSE); si es FALSE, escoge el segundo (TRUE). Una reducción-beta consiste en aplicar una función sustituyendo su parámetro por el argumento, evitando capturar variables libres.

Caso 1: NOT TRUE

1	`(λp. p FALSE TRUE) TRUE`	Expresion inicial.
2	`TRUE FALSE TRUE`	Sustituir p por TRUE (paso beta).
3	`(λx.λy.x) FALSE TRUE`	Expandir la definición de TRUE.
4	`(λy.FALSE) TRUE`	Sustituir x por FALSE.
5	`FALSE`	Sustituir y por TRUE; y no aparece en el cuerpo.

Caso 2: NOT FALSE

```
(λp. p FALSE TRUE) FALSE
→ FALSE FALSE TRUE
→ (λx.λy.y) FALSE TRUE
→ (λy.y) TRUE
→ TRUE
```

Por lo tanto, la definición satisface exactamente la tabla de verdad de la negación: NOT TRUE = FALSE y NOT FALSE = TRUE.

PROBLEMA 1 · 25%

Recursión en programación funcional

La recursión ocurre cuando una función se define en términos de sí misma. Reemplaza con frecuencia a un ciclo imperativo porque permite repetir una transformación sin mantener un contador mutable. Toda recursión correcta debe acercarse a un caso base, que detiene las llamadas, y tener un caso recursivo, que reduce el problema.

Ejemplo: factorial

```
const factorial = n =>
  n <= 1 ? 1 : n * factorial(n - 1);

factorial(4)
= 4 * factorial(3)
= 4 * 3 * factorial(2)
= 4 * 3 * 2 * factorial(1)
= 24
```

Elemento	Funcion	En el ejemplo
Caso base	Detiene la recursión y devuelve un valor directo.	Si n <= 1, devuelve 1.
Caso recursivo	Convierte el problema en otro más pequeño.	Multiplica por factorial(n - 1).
Progreso	Garantiza que se alcanzara el caso base.	n disminuye en cada llamada.

Recursión de cola

Una llamada es de cola cuando la llamada recursiva es la última operación de la función. Esto permite que ciertos lenguajes reutilicen el marco de ejecución. Sin embargo, Node.js no garantiza optimización de llamadas de cola, por lo que una recursión demasiado profunda puede desbordar la pila. Para listas grandes suelen ser más seguros map, filter y reduce, cuyas iteraciones las controla el entorno.

PROBLEMA 1 · 25%

Ciclos en programación funcional

Un ciclo tradicional usa una variable que cambia, por ejemplo i++. El enfoque funcional expresa el resultado deseado mediante funciones de orden superior. La repetición sigue existiendo, pero queda encapsulada y cada etapa produce un valor nuevo.

Equivalencias comunes

Necesidad	Herramienta funcional	Resultado
Transformar cada elemento	map	Lista nueva con igual cantidad de elementos.
Conservar algunos elementos	filter	Lista nueva con los elementos que cumplen una condición.
Combinar toda la lista	reduce	Un valor acumulado, por ejemplo una suma.
Repetir una definición	recursión	El mismo problema aplicado a una entrada menor.

Ejemplo declarativo

```
const numeros = [1, 2, 3, 4, 5, 6];

const sumaDeCuadradosPares = numeros
  .filter(n => n % 2 === 0)
  .map(n => n * n)
  .reduce((acumulado, n) => acumulado + n, 0);

// Resultado: 4 + 16 + 36 = 56
```

La lectura es directa: seleccionar pares, elevarlos al cuadrado y sumarlos. No se cambia numeros, no hay indice mutable y cada lambda describe una transformación pequena. Internamente el lenguaje recorre la lista; por ello, usar funciones de orden superior no significa que el costo de iterar desaparezca.

En los problemas del laboratorio: sort recibe una lambda comparadora; la transpuesta combina dos map; y la eliminación utiliza filter con una lambda predicado.

PROBLEMA 1 · 25%

Cuándo es prudente usar programación funcional

El paradigma funcional es especialmente conveniente cuando el problema puede describirse como transformaciones de datos. Las funciones puras siempre producen el mismo resultado para la misma entrada y no alteran estado externo, lo que simplifica las pruebas y el razonamiento.

Es prudente cuando...	Beneficio
Se procesan colecciones o flujos de datos.	map, filter y reduce expresan la intención con claridad.
Las reglas de negocio deben probarse de forma aislada.	Las funciones puras no dependen de estado oculto.
Se requiere concurrencia o paralelismo.	Los datos inmutables reducen conflictos por estado compartido.
Importa conservar versiones anteriores de los datos.	La inmutabilidad evita cambios accidentales.

Ejemplo apropiado: procesar calificaciones

```
const promedioAprobados = notas => {
  const aprobadas = notas.filter(nota => nota >= 61);
  return aprobadas.reduce((suma, nota) => suma + nota, 0)
    / aprobadas.length;
};
```

Este caso es apropiado porque la entrada se transforma sin modificarse, el cálculo es determinista y cada etapa puede probarse por separado. Si se desea, la función puede reutilizarse en una aplicación web, una herramienta de consola o una prueba automatica.

PROBLEMA 1 · 25%

Cuándo no conviene usarlo como enfoque exclusivo

No todos los problemas mejoran al forzarlos a ser puramente funcionales. Cuando el dominio es esencialmente una secuencia de cambios de estado, ocultar cada cambio detras de copias o abstracciones puede reducir claridad o rendimiento. Esto no prohibe usar funciones: sugiere combinar paradigmas de manera consciente.

Conviene reconsiderarlo cuando...	Riesgo
Existe un ciclo crítico con límites estrictos de tiempo o memoria.	Crear muchas estructuras inmutables puede agregar asignaciones y recolección de basura.
El trabajo consiste principalmente en E/S y estado externo.	Una abstracción pura puede complicar una secuencia naturalmente imperativa.
El equipo no domina el paradigma y la abstracción es innecesaria.	Recursión o composiciones densas pueden dificultar mantenimiento y depuración.
La entrada puede causar recursión muy profunda en JavaScript.	Node.js puede producir un desbordamiento de pila.

Ejemplo poco apropiado: ciclo principal de un videojuego

Un motor de videojuego actualiza posición, velocidad, entrada del usuario, animación y dispositivos muchas veces por segundo. Representar cada fotograma mediante copias completas e inmutables del mundo puede crear presión de memoria y hacer menos evidente el orden de los efectos. En ese nucleo suele ser más práctico actualizar estructuras controladas y reservar funciones puras para calculos como colisiones, puntajes o reglas.

Conclusión. Es prudente aplicar programación funcional donde la pureza y la inmutabilidad hacen el comportamiento más predecible. No es prudente adoptarla como dogma: si el costo de copiar datos, la profundidad recursiva o la naturaleza secuencial de la E/S dominan el problema, un enfoque imperativo o híbrido puede comunicar mejor la solucion.

Referencias conceptuales

Hutton, G. Programming in Haskell, 2.a ed.; y Barendregt, H. P. The Lambda Calculus: Its Syntax and Semantics. Las implementaciones y explicaciones de este documento fueron preparadas específicamente para este laboratorio.